Lecture 2: Generative models for classification

Alex Reinhart

Machine Learning II: 46-927

February 1, 2021

Announcements

- Homework 1 and first step of project are both due Wednesday.
- Note important typo fix on Canvas for problem 2 (e).
- All office hours scheduled and on Canvas, including mine (Wednesday morning)

Today

Last mini you started with regression and introduced classification. Today we will continue on classification ideas and introduce generative models

- Brief review of how to assess classifiers
- Generative models
  - Example: Linear Discriminant Analysis
  - Example: Naive Bayes *(maybe)*

Where we were

We switched from regression to classification.

Now we observe pairs $(X, Y) \sim \mathcal{P}$, with $X \in \mathbb{R}^p$ and $Y \in \{0, 1\}$ (or sometimes $\{-1, 1\}$).

Based on n observed pairs, we estimate a function $\hat{f}$ that "guesses Y well" for future $(X, Y) \sim \mathcal{P}$.

In a classification problem with K classes, we can make $K(K-1)$ different kinds of mistakes! This leads to two different $K \times K$ matrices reflecting classification performance.

Confusion matrix and loss

Confusion Matrix, which tabulates counts of what actually happened.

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	True Positive	False Positive
Guess $Y = 0$	False Negative	True Negative

Loss Matrix, which tabulates how bad each outcome combination is for you.

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	0	L_{01}
Guess $Y = 0$	L_{10}	0

Confusion matrix: error rates

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	True Positive	False Positive
Guess $Y = 0$	False Negative	True Negative

We are interested in many different quantities in the confusion matrix.
The most famous is the misclassification rate: $(FN + FP)/n$

If False positives and False negatives are equally bad, then minimizing the misclassification rate is a good goal.

If we set our loss to have $L_{01} = L_{10} = 1$ and $L_{00} = L_{11} = 0$, minimizing the expected loss will be the same as minimizing the expected misclassification rate.

Statistical decision theory

We showed that the expected misclassification error (or expected prediction error),

$$\mathrm{EPE} = \mathbb{E}\left(\mathcal{L}(Y, \hat{f}(X))\right) = \mathbb{E}\left(\mathbf{1}_{Y \neq f(X)}\right),$$

is minimized by the ideal rule:

$$f(x) \underset{g \in \{1,\ldots,K\}}{\operatorname{argmax}} \mathbb{P}(Y = g | X = x),$$

if we knew $\mathbb{P}(Y = g|X = x)$ for all g. This is called the Bayes Classifier, and the error rate it achieves is the Bayes Rate.

Building a classifier

We showed that the ideal classifier is the Bayes classifier:

$$\begin{aligned}\hat{f}(x) &= \underset{k=1,\ldots K}{\operatorname{argmax}} \ \mathrm{P}(Y = k|X = x) \\ &= \underset{k=1,\ldots K}{\operatorname{argmax}} \ \mathrm{P}(X = x|Y = k) \cdot \pi_k\end{aligned}$$

To build a classifier, we either:

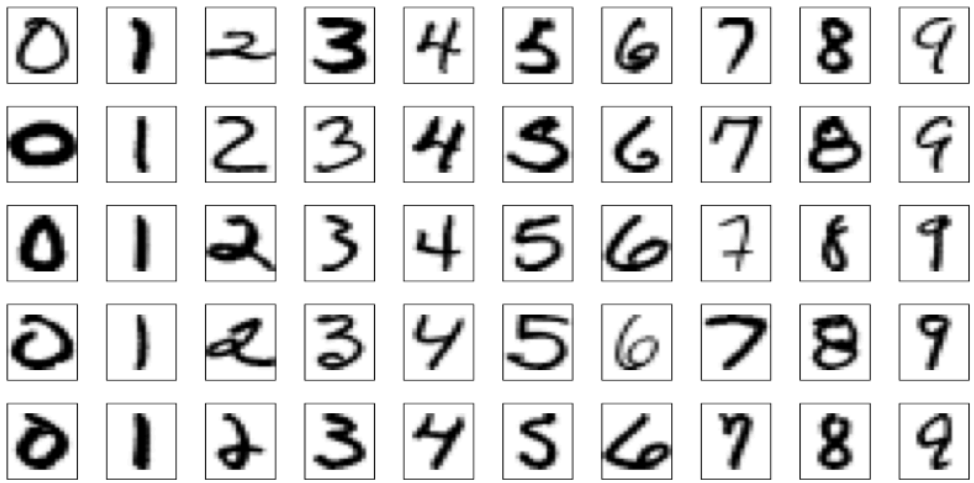
1. Estimate $\mathbb{P}(Y = k|X = x)$ directly. (Discriminative models)

2. **Estimate both π_k and $\mathbb{P}(X = x|Y = k)$ for each class. (Generative models)**

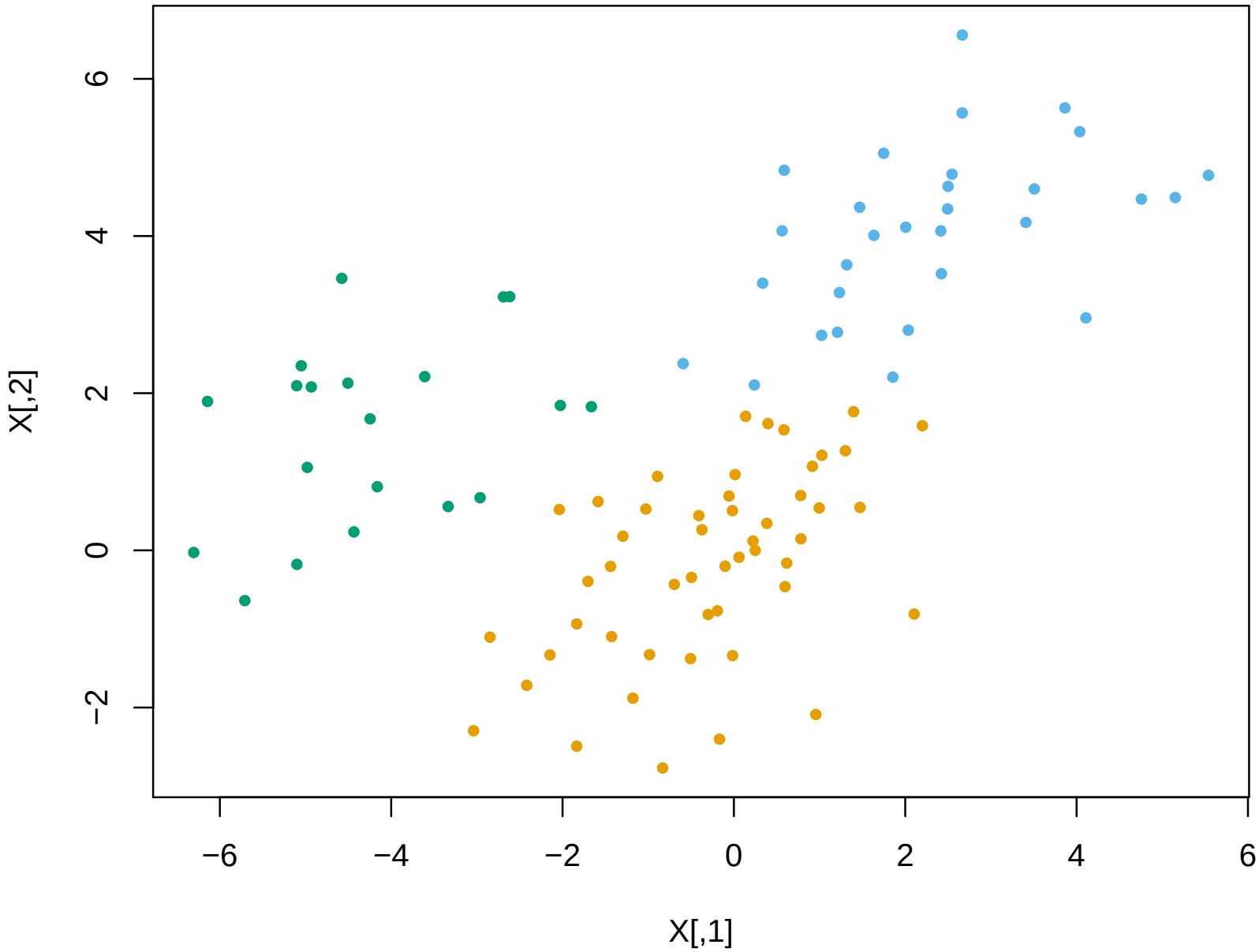
Generative models

Generative models are nice, because they let us think about modeling the process that *generated the data*

Think about the classic MNIST digits data:



It might be easier to describe what a 4 looks like than how all the digits are different.


X[,2]
6
4
2
0
−2
−6
−4
−2
0
2
4
6
X[,1]

Generative models

To implement a generative classifier, we need to decide how to model and estimate π_k and $\mathbb{P}(X = x|Y = k)$. π_k is easy, the distribution of $X|Y = k$ is the main question.

A simple, famous choice is to model the distribution of $X|Y = k$ as multivariate Gaussian.

We will look at this briefly, partly as a demonstration of generative models, and partly because it will give us an excuse to introduce several more general useful ideas.

Sidenote: Generative models and the joint distribution

Linear discriminant analysis

Linear discriminant analysis (LDA) models the data X within each class as being normally distributed:

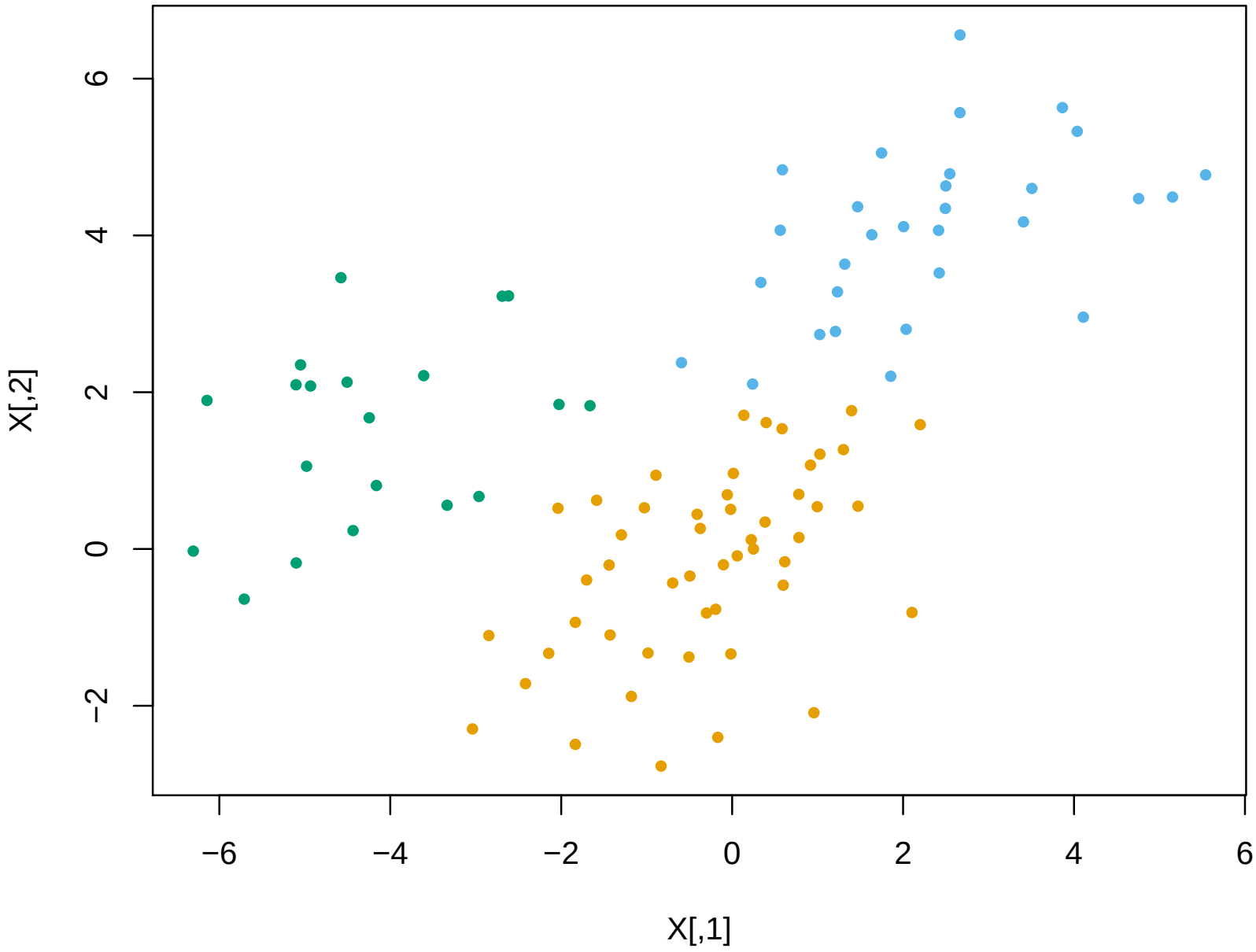
$$h_j(x) = \mathrm{P}(X = x | Y = j) = N(\mu_j, \Sigma) \text{ density}$$

We allow each class to have its own mean $\mu_j \in \mathbb{R}^p$, but we assume a common covariance matrix $\Sigma \in \mathbb{R}^{p \times p}$. Hence

$$h_j(x) = \frac{1}{(2\pi)^{p/2} \det(\Sigma)^{1/2}} \exp\Big\{ -\frac{1}{2}(x - \mu_j)^T \Sigma^{-1} (x - \mu_j) \Big\}$$

Think of this as a model for classification problems where the different classes look like shifted clumps.

See ISL section 4.4 for further reading, or ESL section 4.3 for a mathier version


X[,2]
6
4
2
0
−2
−6
−4
−2
0
2
4
6
X[,1]

Linear discriminant analysis

What does the decision rule look like? We want to find j so that $\mathrm{P}(X = x|Y = j) \cdot \pi_j = h_j(x) \cdot \pi_j$ is the largest. We can define the rule:

$$
\begin{aligned}
f^{\mathrm{LDA}}(x) &= \underset{j=1,\ldots K}{\operatorname{argmax}} \ \frac{1}{(2\pi)^{p/2}\det(\Sigma)^{1/2}} e^{-\frac{1}{2}(x-\mu_j)^T\Sigma^{-1}(x-\mu_j)} \cdot \pi_j \\
&= \underset{j=1,\ldots K}{\operatorname{argmax}} \\
&= \underset{j=1,\ldots K}{\operatorname{argmax}} \\
&= \underset{j=1,\ldots K}{\operatorname{argmax}} \ \delta_j(x)
\end{aligned}
$$

We call $\delta_j(x)$, $j = 1, \ldots K$ the discriminant functions. Note

$$
\delta_j(x) = x^T\Sigma^{-1}\mu_j - \frac{1}{2}\mu_j^T\Sigma^{-1}\mu_j + \log \pi_j
$$

is just an affine function of x

LDA: Estimation

In practice, we have to estimate the parameters π_j, μ_j, and Σ. We estimate them based on the training data $x_i \in \mathbb{R}^p$ and $y_i \in \{1, \dots K\}$, $i = 1, \dots n$, by:

- $\hat{\pi}_j = n_j/n$, the proportion of observations in class j
- $\hat{\mu}_j = \frac{1}{n_j} \sum_{y_i = j} x_i$, the centroid of class j
- $\hat{\Sigma} = \frac{1}{n-K} \sum_{j=1}^{K} \sum_{y_i = j} (x_i - \hat{\mu}_j)(x_i - \hat{\mu}_j)^T$, the pooled sample covariance matrix

(Here n_j is the number of points in class j)

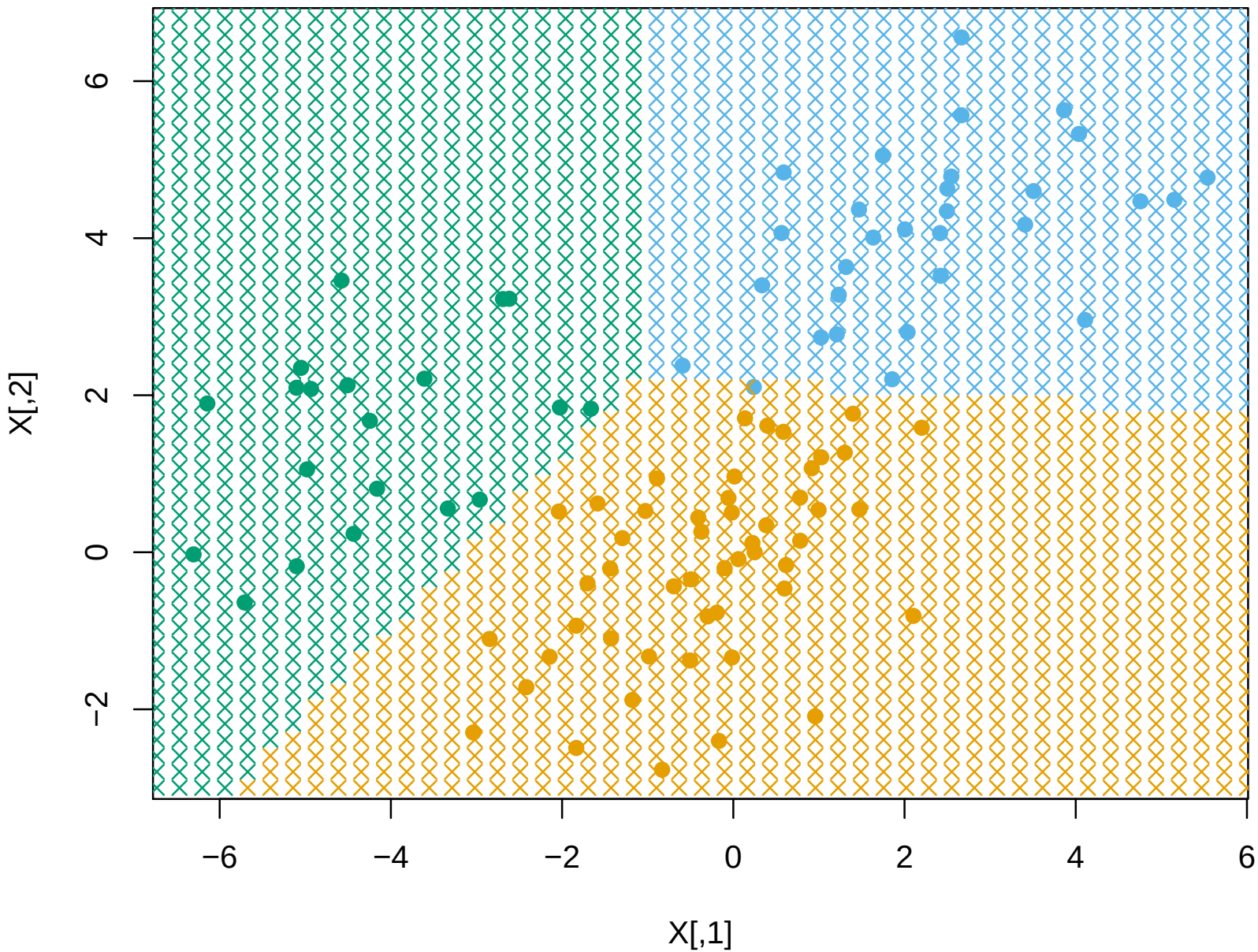
LDA: Estimation

This gives the estimated discriminant functions:

$$\hat{\delta}_j(x) = x^T \hat{\Sigma}^{-1} \hat{\mu}_j - \frac{1}{2} \hat{\mu}_j^T \hat{\Sigma}^{-1} \hat{\mu}_j + \log \hat{\pi}_j$$

and finally the linear discriminant analysis rule for new $x \in \mathbb{R}^p$,

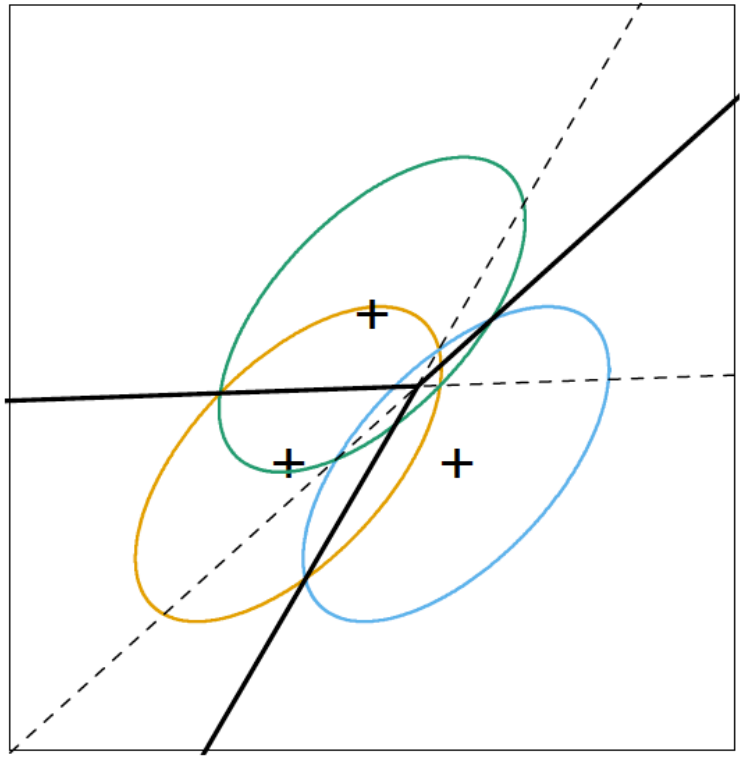
$$\hat{f}^{\text{LDA}}(x) = \underset{j=1,\ldots K}{\text{argmax}} \ \hat{\delta}_j(x)$$

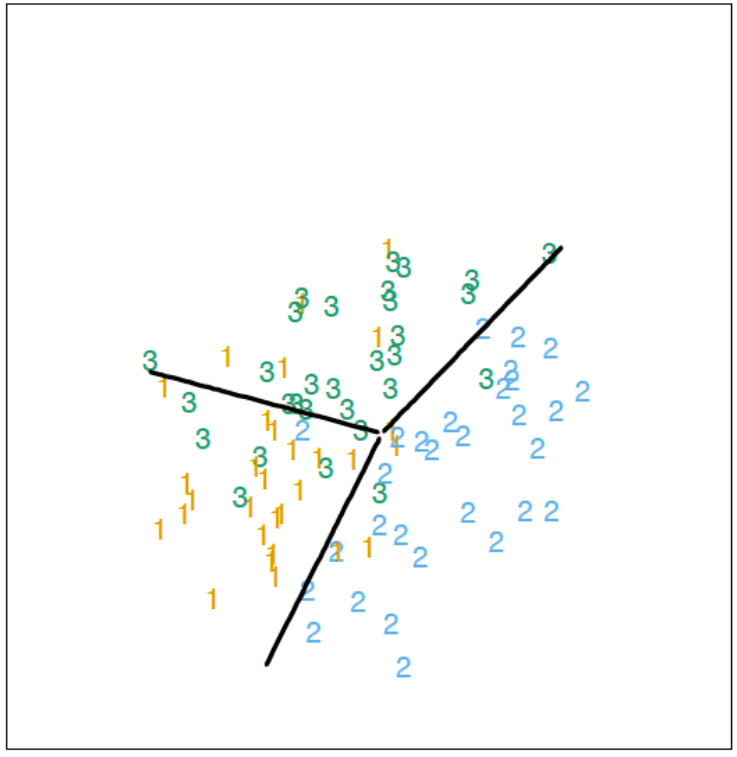

X[,2]
6
4
2
0
−2
−6
−4
−2
0
2
4
6
X[,1]

Example: LDA decision boundaries

Example of decision boundaries from LDA (from ESL page 109):

$f^{\mathrm{LDA}}(x)$ $\hat{f}^{\mathrm{LDA}}(x)$





Are the decision boundaries the same as the perpendicular bisectors between the class centroids?

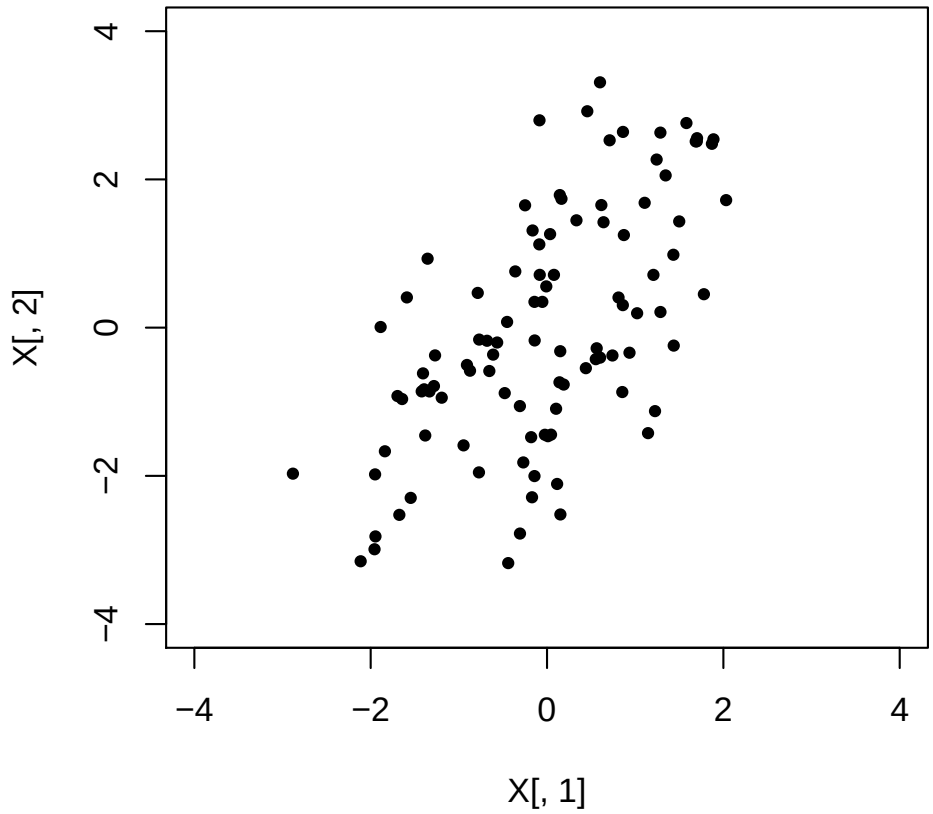
Thinking about the decision function

$$\hat{f}^{\mathrm{LDA}}(x) = \underset{j=1,\ldots K}{\operatorname{argmax}}\, x^T \hat{\Sigma}^{-1} \hat{\mu}_j - \frac{1}{2} \hat{\mu}_j^T \hat{\Sigma}^{-1} \hat{\mu}_j + \log \hat{\pi}_j$$

What changes if our loss function is not 0-1? What changes in the population proportion π_k changes?

Thinking about the decision function

Let's look at the $(x_i - \mu_k)^T \Sigma^{-1} (x_i - \mu_k)$ term that appears in the LDA decison rule. How should we think about this?



Mahalanobis distance

Mahalanobis distance measures the distance from a center in terms of the variance of the distribution.

For a Gaussian, it captures how far out in the tail of the distribution a point is.

This is used for LDA, QDA, and Multivariate Gaussian distributions.

It can also be used for outlier detection and for clustering!

Suppose I've seen a lot of data, and now a new point comes along. How can I tell whether it's a "rare" or outlier point?

Mahalanobis distance, PCA, and LDA

Note that LDA equivalently minimizes over $k = 1, \ldots K$,

$$\frac{1}{2}(x - \hat{\mu}_k)^T \hat{\Sigma}^{-1}(x - \hat{\mu}_k) - \log \hat{\pi}_k$$

It helps to factorize $\hat{\Sigma}$ (i.e., compute its eigendecomposition):

$$\hat{\Sigma} = UDU^T$$

where $U \in \mathbb{R}^{p \times p}$ has orthonormal columns (and rows), and $D = \mathrm{diag}(d_1, \ldots d_p)$ with $d_k \geq 0$ for each k. Then we have $\hat{\Sigma}^{-1} = UD^{-1}U^T$, and

$$(x - \hat{\mu}_k)^T \hat{\Sigma}^{-1}(x - \hat{\mu}_k) =$$

This is just the squared distance between $\tilde{x}$ and $\tilde{\mu}_k$!

LDA transformed

Hence the LDA procedure can be described as:

1. Compute the sample estimates $\hat{\pi}_k, \hat{\mu}_k, \hat{\Sigma}$
2. Factor $\hat{\Sigma}$, as in $\hat{\Sigma} = UDU^T$
3. Transform the class centroids $\tilde{\mu}_k = D^{-1/2}U^T\hat{\mu}_k$
4. Given any point $x \in \mathbb{R}^p$, transform to $\tilde{x} = D^{-1/2}U^T x \in \mathbb{R}^p$, and then classify according to the nearest centroid in the transformed space, adjusting for class proportions—this is the class k for which $\frac{1}{2}\|\tilde{x} - \tilde{\mu}_k\|_2^2 - \log \hat{\pi}_k$ is smallest

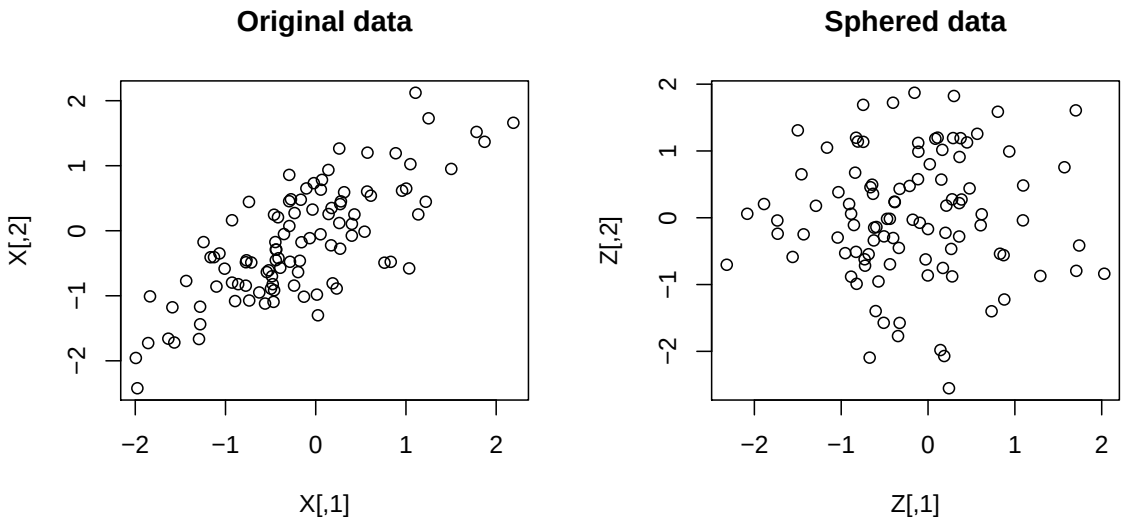
Sphering

What is this transformation doing? Think about applying it to the observations:

$$\tilde{x}_i = D^{-1/2}U^T x_i, \quad i = 1, \ldots n$$

This is basically sphering the data points, because if we think of $x \in \mathbb{R}^p$ were a random variable with covariance matrix $\hat{\Sigma}$, then

$$\mathrm{Cov}(D^{-1/2}U^T x) =$$



Linear subspace spanned by sphered centroids

LDA compares the quantity $\frac{1}{2}\|\tilde{x} - \tilde{\mu}_k\|_2^2 - \log \hat{\pi}_k$ across the classes $k = 1, \ldots K$.
Think about the $K - 1$ dimensional plane that goes through all K centroids, $\tilde{\mu}_1, \ldots, \tilde{\mu}_K$.

Because we are looking at distance to these centroids, only the distance projected onto this plane matters!

LDA procedure summarized

We can think of the LDA procedure as:

1. Compute the sample estimates $\hat{\pi}_k, \hat{\mu}_k, \hat{\Sigma}$

2. Make two transformations: first, sphere the data points, based on factoring $\hat{\Sigma}$; second, project down to the affine subspace spanned by the sphered centroids. This can all be summarized a single linear transformation $A \in \mathbb{R}^{(K-1)\times p}$

3. Given any point $x \in \mathbb{R}^p$, transform to $\tilde{x} = Ax \in \mathbb{R}^{K-1}$, and classify according to the class $k = 1, \ldots K$ for which

$$\frac{1}{2}\|\tilde{x} - \tilde{\mu}_k\|_2^2 - \log \hat{\pi}_k$$

is smallest, where $\tilde{\mu}_k = A\hat{\mu}_k$

This way of describing LDA may sound more complicated, but actually, it's much simpler! After applying A, we've reduced the problem from p to $K-1$ dimensions, and then it's basically nearest centroid classification:

$$\hat{f}^{\mathrm{LDA}}(x) = \underset{k=1,\ldots K}{\operatorname{argmin}} \ \frac{1}{2}\|\tilde{x} - \tilde{\mu}_k\|_2^2 - \log \hat{\pi}_k$$

(The only distinction being that we adjust for class proportions)

How do these linear classifiers compare?

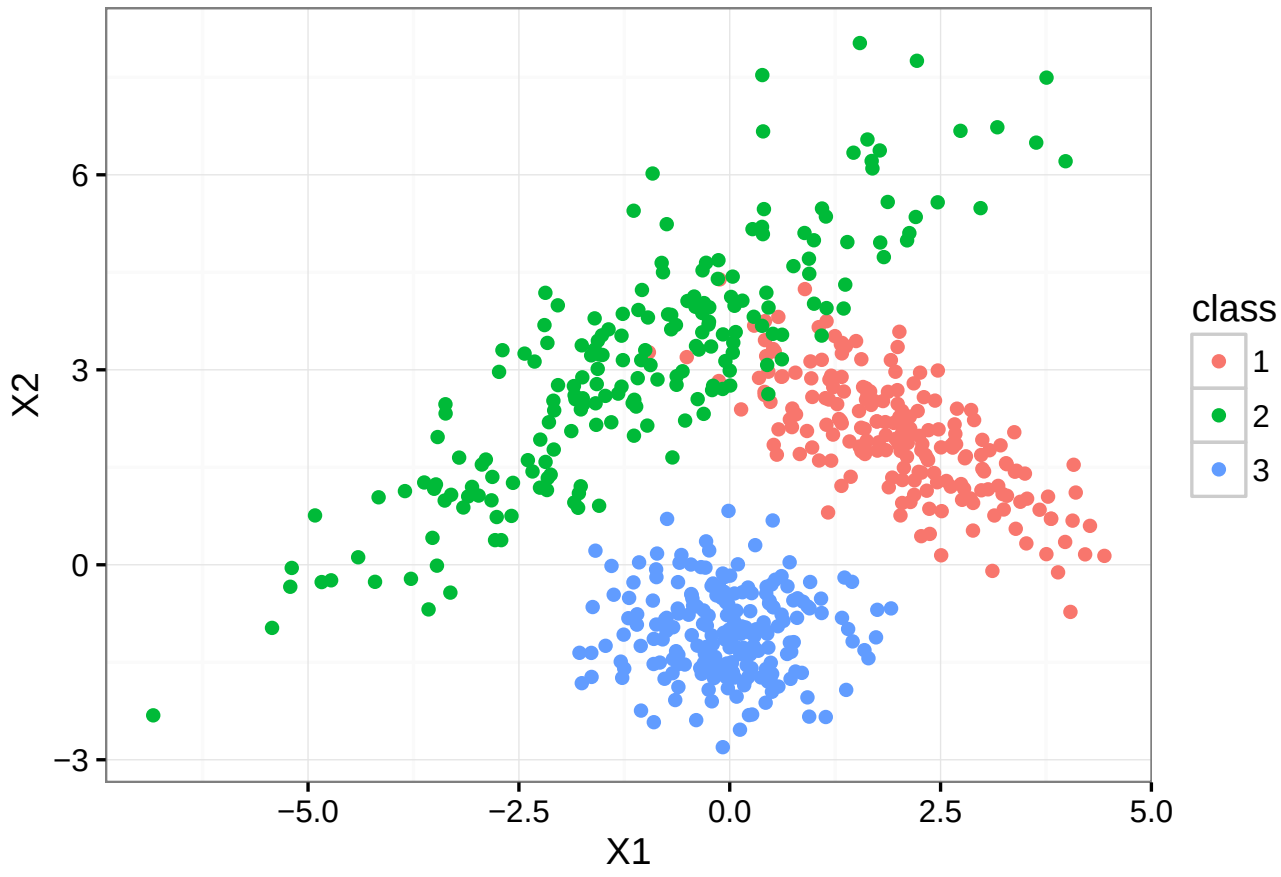
Though the decision rules are both linear, these methods produce different classifications because they rely on different assumptions.

1. LDA: Works well if the groups are in "clumps" so that the Gaussian distribution is reasonable. Also assumes the shapes are similar.

2. Logistic: Only requires that $\log \frac{P(Y=1|X=x)}{P(Y=0|X=x)} \approx x_i^T \beta$, which is strictly weaker. LDA will do better when its assumptions are reasonable, but otherwise worse. Logistic focuses more on the boundary points.

Variations on LDA: unequal Σ

The LDA model assumes that our covariance is the same for all groups. What if it's not?



$$\Sigma_1 = \begin{pmatrix} 1 & -0.7 \\ -0.7 & 1 \end{pmatrix}, \Sigma_2 = \begin{pmatrix} 3 & 2.5 \\ 2.5 & 3 \end{pmatrix}, \Sigma_3 = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \end{pmatrix}$$

All the same math goes through, giving Quadratic Discriminant Functions (and QDA)

$$\begin{aligned}\delta_k(x) &= -\frac{1}{2}(x-\mu_k)^T\Sigma^{-1}(x-\mu_k) - \frac{1}{2}\log|\Sigma_k| + \log\pi_k \\ &= -\frac{1}{2}x^T\Sigma_k^{-1}x + \mu_k\Sigma_k^{-1}x - \frac{1}{2}\mu_k^T\Sigma_k^{-1}\mu_k - \frac{1}{2}\log|\Sigma_k| + \log\pi_k\end{aligned}$$

The Σ matrices are now different in each function are now different, and the boundaries $\delta_k(x) > \delta_{k'}$ are no longer linear. Instead, they are

This allows the boundaries to curve around groups to account for different patterns of spread. It comes at a cost though:

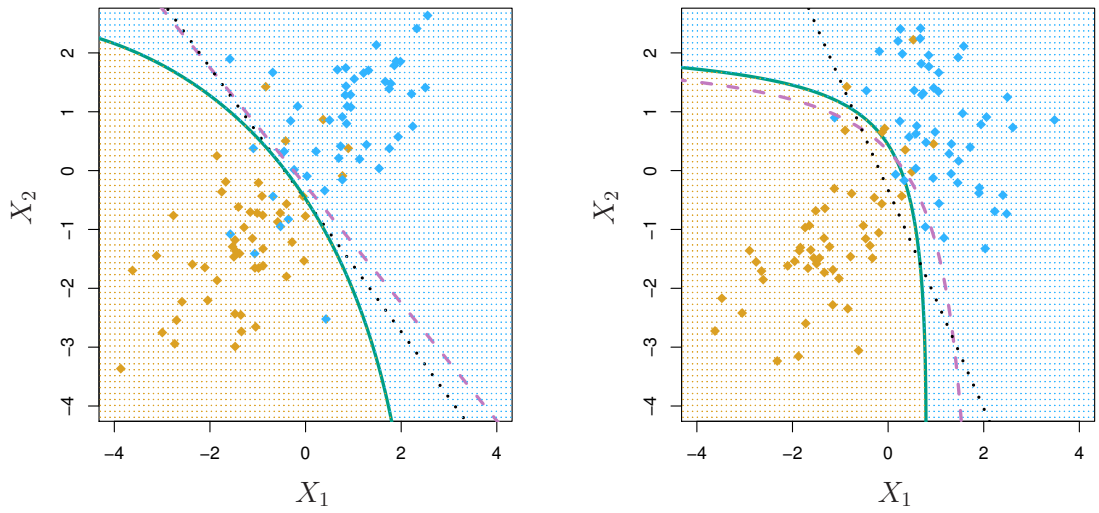
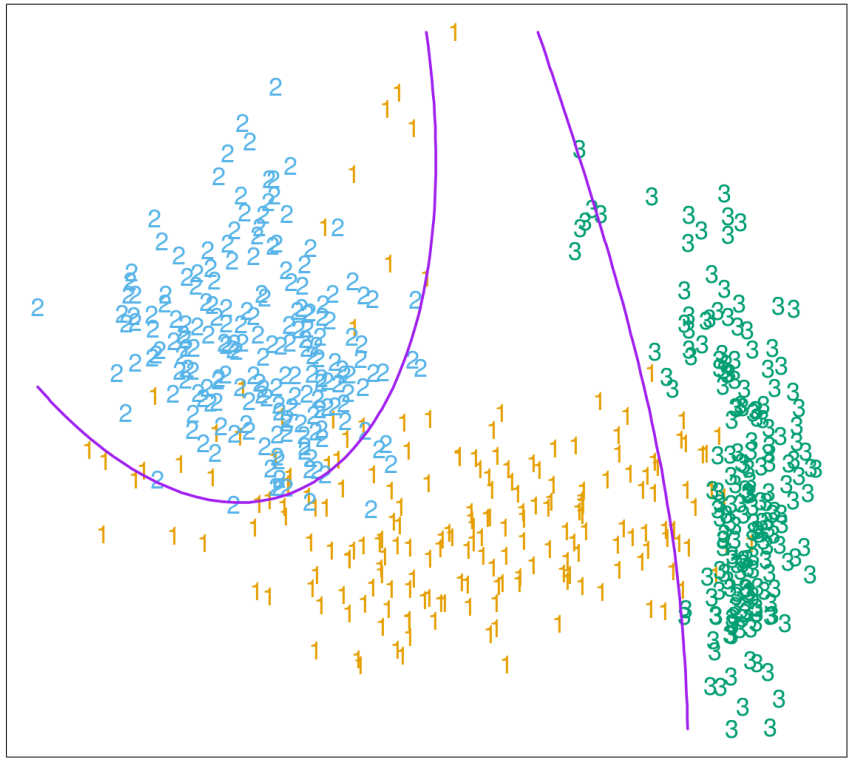


Figure 4.9 from ISL. Dashed purple curve is the Bayes classifier decision boundary. Solid green curve is QDA, dotted black line is LDA. Left: True boundary is linear. Right: True boundary is quadratic

Variations on LDA: unequal Σ

Quadratic discriminant analysis (QDA):



(ESL 4.6)

Variations on LDA: high dimensions

LDA really becomes instructive when we consider its performance over a range of dimensions.

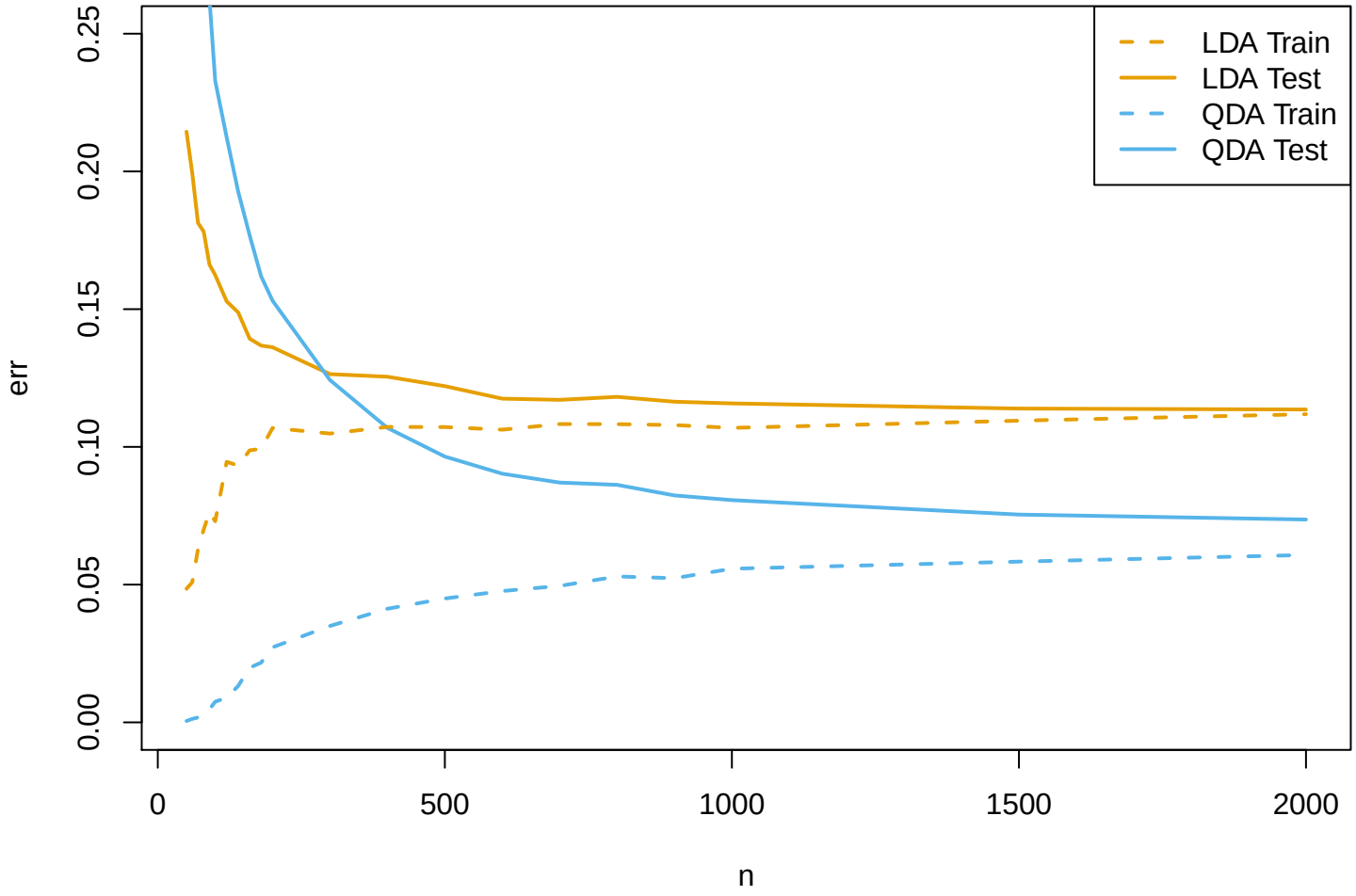
Fitting QDA requires estimating

Fitting LDA requires estimating

As the number of parameters increases, the of our estimator increases (but the hopefully decreases).

Suppose that we have two groups, drawn respectively from $N(\mu_1, \Sigma_1)$ and $N(\mu_2, \Sigma_2)$. If we choose to use LDA instead of QDA, we are choosing a more biased model to reduce variance.

Variations on LDA: high dimensions



Variations on LDA: high dimensions

Suppose the dimension gets even higher?

What if I can't even estimate the group means well?

Naive Bayes

Imagine that you have $n = 2000$ observations and $p = 1000$ features.

It will be incredibly hard to estimate $f_k = P(X = x|Y = k)$ well for any complicated model!

You need very strong "assumptions" on f_k (reducing parameters/variance)!

Naive Bayes *assumes* (well, models) that all of the components of $X = (X_1, \ldots, X_p)$ are *independent* (conditional on Y).

Naive Bayes

Naive Bayes models all of the components of $X = (X_1, \ldots, X_p)$ as *independent* (conditional on Y).

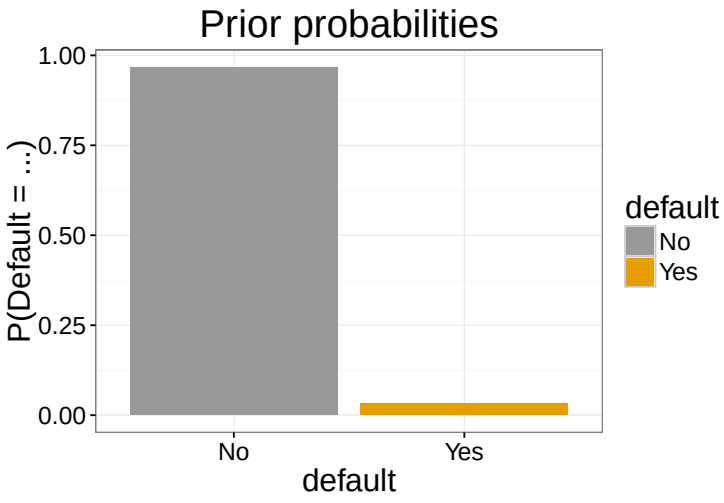
Under this independence assumption, the class distributions factor!

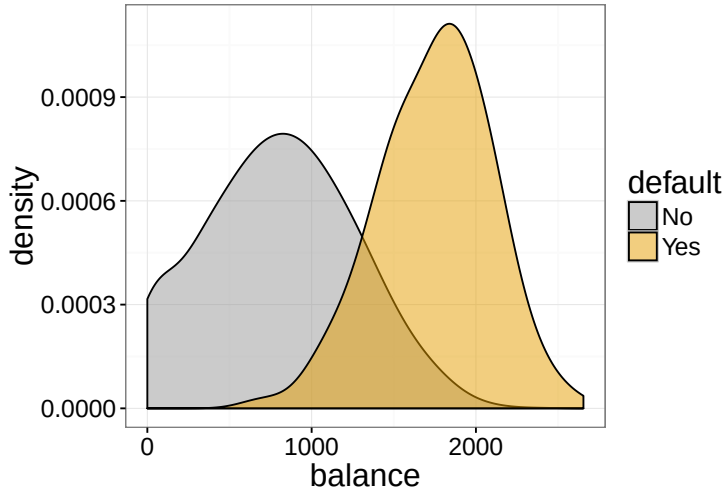
$$
\begin{aligned}
f_k(x) &= P(X = x | Y = k) \\
&= \\
&= \\
&=
\end{aligned}
$$

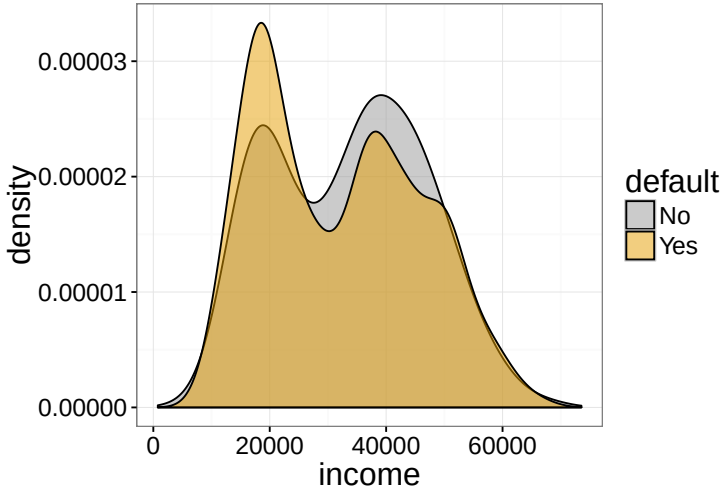
This means that we can just estimate the *univariate* distributions!

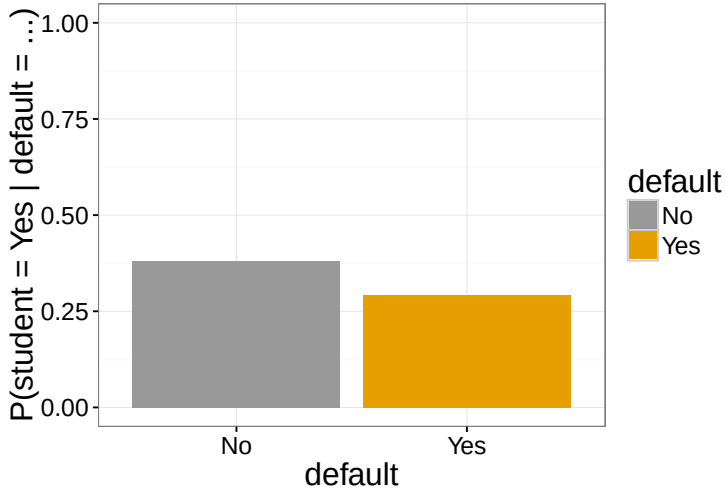
$$f_k^{(j)}(x_j) = P(X_j = x_j | Y = k)$$

Example: Default data from ISLR









We can easily calculate these simplified class distributions:

$$\hat{f}_{\text{Yes}} = \hat{f}_{\text{Yes}}(\text{income})\hat{f}_{\text{Yes}}(\text{balance})\hat{f}_{\text{Yes}}(\text{student})$$
$$\hat{f}_{\text{No}} = \hat{f}_{\text{No}}(\text{income})\hat{f}_{\text{No}}(\text{balance})\hat{f}_{\text{No}}(\text{student})$$

And then plug them into the Bayes classifier formula, just like we did for LDA

$$P(\text{default}|i, b, s) = \frac{\hat{\pi}_{\text{Yes}}\hat{f}_{\text{Yes}}(i, b, s)}{\hat{\pi}_{\text{Yes}}\hat{f}_{\text{Yes}}(i, b, s) + \hat{\pi}_{\text{No}}\hat{f}_{\text{No}}(i, b, s)}$$

Naive Bayes

Naive Bayes scales well to problems with very large p. We only need enough data to estimate each of the marginal distributions well.

It also allows you to have a flexible choice of models for each of the univariate distributions.

However, Naive Bayes cannot capture *interactions* between the features within each class!

LDA and QDA are able to incorporate these feature interactions, at the cost of needing to estimate them.